

Track C: Microsoft Fabric

Generated from the same content files as the app. Each module lists its roadmap months. Solutions are in the appendix.

Contents

Track C: Microsoft Fabric	2
C2 Lakehouse, Warehouse and SQL analytics endpoint	3
C2-L3 Lakehouse, Warehouse or SQL analytics endpoint.....	3
Track C best-practice checklist	9
Appendix: solutions and answer keys	10

Track C: Microsoft Fabric

Unify data, analytics and Copilot in Fabric, and verify what Copilot produces.

C2 Lakehouse, Warehouse and SQL analytics endpoint

Roadmap: Months 16 to 18: MLOps and deployment; Current work. When to use each.

C2-L3 Lakehouse, Warehouse or SQL analytics endpoint

Concept

Fabric gives you three ways to hold and query tables. All three store data as Delta tables in OneLake. They differ in who writes the data and in which language.

Lakehouse. You write with Spark: notebooks, Spark job definitions, Dataflows Gen2 or pipelines. It holds files and Delta tables side by side. Choose it for raw files, PySpark transformations and data science. This is the closest match to your Databricks work.

SQL analytics endpoint. Every lakehouse gets one automatically. It gives a **read-only** T-SQL surface over the lakehouse's Delta tables. You can `SELECT`, and you can create views, functions and stored procedures. You cannot `INSERT`, `UPDATE` or `DELETE` table data through it. It picks up new or changed tables through a metadata sync. Microsoft documents the usual lag as under one minute, but it varies.

Warehouse. You write with T-SQL. It supports full DML and DDL, and multi-table transactions. Choose it when a team works in T-SQL, loads data with stored procedures, or needs several tables to change together or not at all.

One rule decides most cases: **who writes the data, and in which language?** Spark writers go to a lakehouse. T-SQL writers go to a warehouse. T-SQL readers of lakehouse data use the SQL analytics endpoint.

You do not have to copy data between them. A warehouse query reads lakehouse tables in the same workspace with a three-part name: `lakehouse_name.schema.table`. Direct Lake semantic models read from either.

Check the status of each feature you rely on in `VERSIONS.md` before you build. Fabric changes monthly.

Why it matters

A team moves a finance reconciliation from SQL Server to Fabric. They land every table in a lakehouse, because the Databricks team already uses lakehouses. The month-end stored procedure updates three tables. On the SQL analytics endpoint it fails on its first `UPDATE`, because the endpoint is read-only.

Under deadline, someone rewrites it as three separate Spark notebook writes. On one month-end the second notebook fails. Table one holds the new month and table three holds the old one, and the reconciliation report shows a balance that does not tie. A warehouse transaction would have rolled back all three tables together.

Worked example: weak code

```
❶ -- Connected to the SQL analytics endpoint of lakehouse lh_finance
❷ CREATE PROCEDURE dbo.close_month @period CHAR(7) AS
❸ BEGIN
❹ UPDATE dbo.ledger SET status = 'closed' WHERE period = @period;
❺ INSERT INTO dbo.balances
❻ SELECT account_id, @period, SUM(amount) FROM dbo.ledger
❼ WHERE period = @period GROUP BY account_id;
❽ DELETE FROM dbo.open_items WHERE period = @period;
❾ END;
```

1. **Line 1.** The endpoint gives read-only access to lakehouse tables. Every write below fails.
2. **Line 4.** `UPDATE`, `INSERT` and `DELETE` on lakehouse tables are not supported through the endpoint.
3. **Line 8.** Three tables must change together. Without a transaction, a failure halfway leaves them inconsistent.

Worked example: strong code

```
❶ -- Connected to warehouse wh_finance
❷ CREATE PROCEDURE dbo.close_month @period CHAR(7) AS
❸ BEGIN
❹ BEGIN TRANSACTION;
❺ UPDATE dbo.ledger SET status = 'closed' WHERE period = @period;
❻ INSERT INTO dbo.balances (account_id, period, balance)
❼ SELECT account_id, @period, SUM(amount) FROM dbo.ledger
❽ WHERE period = @period GROUP BY account_id;
❾ DELETE FROM dbo.open_items WHERE period = @period;
❿ COMMIT TRANSACTION;
⓫ END;
```

1. **Line 1.** The T-SQL writer works in a warehouse, which supports full DML.

2. **Line 4.** All three changes commit together or not at all. Warehouses support multi-table transactions.
3. **Line 6.** Name the target columns. An INSERT without a column list breaks when someone adds a column.

The data and the logic are the same. The item type decides whether the procedure runs at all, and whether a failure leaves the tables consistent.

Recap card

- **Lakehouse, Warehouse or endpoint: the deciding question** Who writes the data, and in which language? Spark writes to a lakehouse, T-SQL writes to a warehouse.
- **What can you do on the SQL analytics endpoint?** SELECT and create views, functions and procedures. No INSERT, UPDATE or DELETE on table data.
- **When do you need a Warehouse over a Lakehouse?** T-SQL writes, stored procedure loads, and multi-table transactions.
- **Read a lakehouse table from a warehouse in the same workspace** Use a three-part name: lakehouse.schema.table. No copy.
- **A new lakehouse table is missing from the endpoint** Metadata sync lag. Usually under a minute. Refresh or retry before dependent steps.

Practice

Exercise 1. Multiple choice (level 1)

Which statement about a lakehouse's SQL analytics endpoint is true?

- a) You can run UPDATE on lakehouse tables through it.
- b) You can create a view over lakehouse tables in it.
- c) You create it by hand after you create the lakehouse.
- d) It stores its own copy of the lakehouse tables.

Why?

- r1) Views are T-SQL objects that read data. The endpoint allows objects that read, not statements that change table data.
- r2) Views are stored inside the Delta tables.

Exercise 2. Architecture design (level 2)

Pick the Fabric item for each workload.

Options: Lakehouse, Warehouse, SQL analytics endpoint.

1. Land raw JSON survey deliveries and clean them with PySpark
2. Finance loads budgets with T-SQL stored procedures that change several tables together
3. Analysts query the cleaned lakehouse tables with T-SQL and save a few views

Exercise 3. Spot the bug (level 3)

This script runs against the SQL analytics endpoint of `lh_research`. It fails partway. Tap the failing line, then rewrite the script so it runs on the endpoint.

```
1 -- Connected to the SQL analytics endpoint of lh_research
2 CREATE VIEW dbo.v_wave_reach AS
3 SELECT wave_id, SUM(weighted_reach) / SUM(weighted_base) AS reach_pct
4 FROM dbo.nat_metrics
5 GROUP BY wave_id;
6 GO
7 UPDATE dbo.nat_metrics SET weighted_reach = 0 WHERE wave_id = 'W00';
```

Write down the line number of the bug. Then write the fix.

Exercise 4. Write the function (level 3)

On the SQL analytics endpoint, write a view `dbo.v_wave_reach` with one row per `wave_id` and a column `reach_pct`: the sum of `weighted_reach` divided by the sum of `weighted_base` from `dbo.nat_metrics`. A wave with a zero base must return NULL, not a divide-by-zero error.

Starter code:

```
1 CREATE VIEW dbo.v_wave_reach AS
```

Exercise 5. Fill the gap (level 4)

You are connected to warehouse `wh_finance`. Lakehouse `lh_research` sits in the same workspace. Complete the query so it reads the lakehouse table without copying it.

```
1 INSERT INTO dbo.wave_summary (wave_id, weighted_base)
2 SELECT wave_id, SUM(weighted_base)
3 FROM _____.dbo.nat_metrics
4 GROUP BY wave_id;
```

Exercise 6. Multiple choice (level 5)

A pipeline runs a Spark notebook that creates a new gold table in `lh_research`. The next pipeline step immediately queries the table through

the SQL analytics endpoint and fails with "Invalid object name". An hour later the same query works. What is the most likely cause?

- a) The notebook wrote the table in a format the endpoint cannot read.
- b) The endpoint had not yet synced the new table's metadata.
- c) The endpoint needs a manual refresh every time a table changes.
- d) The pipeline's identity lacks permission on new tables for one hour.

Why?

- r1) The endpoint discovers new Delta tables through an automatic sync that usually takes under a minute but has no fixed limit.
- r2) The endpoint copies data from the lakehouse on a schedule.

Practical: Lakehouse, endpoint and warehouse side by side

Platform: Fabric trial workspace. Run this in a Fabric trial workspace. You create a lakehouse and a warehouse, write one table with Spark, read it through the endpoint, and see a write fail there.

1. In your trial workspace, create a lakehouse named `lh_research`.
2. Open a new notebook attached to `lh_research`. Run:

```
spark.createDataFrame([("W01", 120.0, 400.0), ("W02", 0.0, 0.0)],  
"wave_id STRING, weighted_reach DOUBLE, weighted_base  
DOUBLE").write.mode("overwrite").saveAsTable("nat_metrics").
```
3. Switch the lakehouse view to its SQL analytics endpoint. Wait for `nat_metrics` to appear. Note how long it takes.
4. Open a new SQL query on the endpoint and create the `dbo.v_wave_reach` view from exercise 4. Query it. Wave W02 returns NULL.
5. In the same query window, run `UPDATE dbo.nat_metrics SET weighted_reach = 0 WHERE wave_id = 'W01';` and read the error message.
6. Create a warehouse named `wh_finance`. In a new query, run `SELECT * FROM lh_research.dbo.nat_metrics;`. If it fails, record the error, since this lesson marks cross-item queries as unverified.
7. Write down what you saw in each step. Compare it with the recap card.

Sources

- The SQL analytics endpoint is read-only for Delta table data. No INSERT, UPDATE or DELETE. (search-only: <https://learn.microsoft.com/en-us/fabric/data-engineering/lakehouse-sql-analytics-endpoint>)

- You can create views, functions and stored procedures in the SQL analytics endpoint. (search-only: <https://learn.microsoft.com/en-us/fabric/data-engineering/lakehouse-sql-analytics-endpoint>)
- Fabric Warehouse supports full DML, DDL and multi-table transactions. (search-only: <https://learn.microsoft.com/en-us/fabric/fundamentals/decision-guide-lakehouse-warehouse>)
- Metadata sync lag is usually under one minute, and you can refresh it from the portal, the REST API or `sys.sp_dw_refresh_ext_table`. (search-only: <https://learn.microsoft.com/en-us/fabric/data-engineering/sql-analytics-endpoint-metadata-sync>)
- A warehouse reads lakehouse tables in the same workspace with three-part names. (unverified: Check Microsoft Learn: cross-database queries in Fabric Data Warehouse.)

Track C best-practice checklist

- ☐ Check the status (GA or preview) of every Fabric feature before you depend on it.
- ☐ One copy of each dataset. Use shortcuts or mirroring before you copy.
- ☐ Lakehouse for Spark and files. Warehouse for T-SQL writes and multi-table transactions.
- ☐ The SQL analytics endpoint is read-only for table data. Write through Spark or a Warehouse.
- ☐ Semantic models use star schemas. DAX measures state their filter context.
- ☐ Direct Lake models are checked for fallback to DirectQuery.
- ☐ Every Copilot output is read, run on known data and compared against a second method.
- ☐ Workspaces map to domains. Access uses roles and OneLake security, not item sharing.
- ☐ Items live in Git. Changes move through deployment pipelines.
- ☐ Capacity use is monitored, and heavy jobs are scheduled off peak.

Appendix: solutions and answer keys

Attempt each exercise before you read its solution. Track D answer keys are here too.

C2-L3 Lakehouse, Warehouse or SQL analytics endpoint

C2-L3 exercise 1

Hints. (1) The endpoint is described as read-only. Read-only for what? (2) It blocks changes to table data, not every kind of object. (3) Views, functions and stored procedures that read are allowed.

Answer. b) You can create a view over lakehouse tables in it. Because: r1) Views are T-SQL objects that read data. The endpoint allows objects that read, not statements that change table data.

The endpoint is read-only for table data, but you can create views, functions and stored procedures in it. Fabric creates it with the lakehouse, and it reads the same Delta tables in OneLake.

Common wrong answers.

- a: The endpoint is read-only for table data. Write with Spark in the lakehouse, or move the table to a warehouse.
- c: Fabric creates the endpoint automatically with every lakehouse.
- d: It reads the same Delta tables in OneLake. There is no second copy. It syncs metadata, not data.
- b | r2: Right choice, wrong reason. A view is a T-SQL object in the endpoint. The Delta tables stay unchanged.

C2-L3 exercise 2

Hints. (1) Ask who writes the data, and in which language. (2) Spark writers and files: one item. T-SQL writers: another. (3) T-SQL readers of lakehouse data need no new item at all.

Answer.

- Raw files and PySpark: **Lakehouse**. Spark writes, and files sit next to tables.
- T-SQL writes across several tables: **Warehouse**, for DML and multi-table transactions.

- T-SQL reads of lakehouse data, plus views: **SQL analytics endpoint**. No copy needed.

Spark writers use a lakehouse. T-SQL writers use a warehouse. T-SQL readers of lakehouse data use the endpoint that comes with the lakehouse.

Common wrong answers.

- finance = lakehouse: The finance team writes with T-SQL. A lakehouse accepts writes from Spark, and its endpoint is read-only. Use a warehouse.
- finance = endpoint: The endpoint cannot change table data, so the stored procedures would fail on their first UPDATE or INSERT.
- raw = warehouse: A warehouse does not hold raw files, and the team writes with PySpark. Use a lakehouse.
- adhoc = warehouse: Copying the lakehouse tables into a warehouse makes a second copy to keep in sync. The endpoint reads them where they are.

C2-L3 exercise 3

Hints. (1) Which statements change table data? (2) The endpoint accepts reads and objects that read. (3) UPDATE on a lakehouse table fails through the endpoint.

Answer. The bug is on line 7.

Fixed version:

```
1 -- Connected to the SQL analytics endpoint of lh_research
2 CREATE VIEW dbo.v_wave_reach AS
3 SELECT wave_id, SUM(weighted_reach) / SUM(weighted_base) AS reach_pct
4 FROM dbo.nat_metrics
5 GROUP BY wave_id;
6 GO
7 -- Fix wave W00 in the lakehouse with Spark, not here.
```

Line 7 is an UPDATE on a lakehouse table, which the read-only endpoint rejects. Make that correction in a Spark notebook against the lakehouse, and keep only the view here.

Common wrong answers.

- Line 2: Creating a view is allowed on the endpoint. Look for a statement that changes table data.
- Line 3: The division could hit zero, but that is not what stops this script. Look for a write.

C2-L3 exercise 4

Hints. (1) Aggregate both sums per wave, then divide. (2) `NULLIF(x, 0)` returns NULL when x is 0, and dividing by NULL gives NULL. (3) `SUM(weighted_reach) / NULLIF(SUM(weighted_base), 0) AS reach_pct`, then `GROUP BY wave_id`.

Answer.

```
1 CREATE VIEW dbo.v_wave_reach AS
2 SELECT
3     wave_id,
4     SUM(weighted_reach) / NULLIF(SUM(weighted_base), 0) AS reach_pct
5 FROM dbo.nat_metrics
6 GROUP BY wave_id;
```

Sum both columns per wave and divide by `NULLIF(SUM(weighted_base), 0)`. A zero base becomes NULL, so the division returns NULL instead of an error.

C2-L3 exercise 5

Hints. (1) Three-part names are `item.schema.table`. (2) The first part names the item that holds the table. (3) The table sits in the lakehouse `lh_research`.

Answer.

```
1 INSERT INTO dbo.wave_summary (wave_id, weighted_base)
2 SELECT wave_id, SUM(weighted_base)
3 FROM lh_research.dbo.nat_metrics
4 GROUP BY wave_id;
```

A warehouse query reads a lakehouse table in the same workspace with `lh_research.dbo.nat_metrics`. The data stays in one place.

Common wrong answers.

- `g1 = wh_finance: wh_finance` is the warehouse you are in. The table lives in the lakehouse, so name the lakehouse.
- `g1 = OneLake`: There is no database called OneLake. The first part of a three-part name is the item name: the lakehouse.

C2-L3 exercise 6

Hints. (1) The query failed first and worked later, with no change in between. (2) How does the endpoint learn that a new table exists? (3) Look at the metadata sync between the lakehouse and its endpoint.

Answer. b) The endpoint had not yet synced the new table's metadata. Because: r1) The endpoint discovers new Delta tables through an automatic sync that usually takes under a minute but has no fixed limit.

The endpoint syncs lakehouse metadata automatically, usually in under a minute, but the lag varies. When a later step depends on a new table, add a metadata refresh step (portal, REST API) or a retry before the query. Check the current refresh options on Microsoft Learn.

Common wrong answers.

- **a**: The query worked an hour later without any change to the table, so the format was fine.
- **c**: The sync runs automatically. A manual refresh speeds it up, but it is not required every time.
- **d**: Permissions do not change on a timer. The delay points to the metadata sync.
- **b|r2**: Right choice, wrong reason. The endpoint does not copy data. It syncs metadata about the Delta tables.